Name: _____
Surname: _____
Date: _____

Instructions

Always assume each code is run independently, so variables do not persist between exercises.

Questions 1-6: Multiple Choice (5 points each)

On the table below, mark the correct answer for each question. Use an X to mark your choice. If you want to change your answer, fill the entire box and mark a new one. If multiple boxes are marked for the same question, no points will be awarded. See the examples below. Each question has **only one** correct answer. A correct answer awards 5 points. There is no penalty for wrong choices.

Example	A	B	C	D
Chosen A	X	☐	☐	☐
Chosen C, cancelled A	■	☐	X	☐
Chosen B, cancelled A and C	■	X	■	☐
Invalid (multiple choices)	X	X	☐	☐
Invalid (no choices)	■	☐	☐	☐

Questions 7-9: Debugging (10 points each)

These exercises show code cells with one or more errors. You have to find these errors, explain why they happen, and suggest a fix to them.

Questions 10-11: Open Problems (20 points each)

These are open-ended tasks. Explain your approach, outline algorithmic steps, and optionally provide compact working code snippets.

Mark your answers for Questions 1–6 below

Question	A	B	C	D
Q1	☐	☐	☐	☐
Q2	☐	☐	☐	☐
Q3	☐	☐	☐	☐
Q4	☐	☐	☐	☐
Q5	☐	☐	☐	☐
Q6	☐	☐	☐	☐

1. Which of the following statements about for and while loops in Python is TRUE?

   A. A while loop must always terminate after a fixed number of iterations.

   B. A while loop continues until its condition becomes `False`, regardless of how many iterations it takes.

   C. A for loop cannot be exited early using `break`.

   D. A for loop can be used only when the number of iterations is known in advance.

2. After running the following code, what will be the final value of variable "str2"?

```
str1 = "I love strings"

str_list = [char.upper() for char in str1 if char != " "]

str2 = "".join(str_list)
```

   A. `str2 = ‘‘I L O V E S T R I N G S’’`

   B. `str2 = ‘‘I LOVE STRINGS’’`

   C. The code will raise an error.

   D. `str2 = ‘‘ILOVESTRINGS’’`

3. Why will the following code cell fail?

```
1  import numpy as np
2
3  arr = np.arange(4)
4  # Return evenly spaced values within a given interval.
5
6  arr_bool = np.array([True, False, True, False, True])
7  selected = arr[arr_bool]
```

   A. (Line 7) We cannot index an array (`arr`) with a boolean array (`arr_bool`).

   B. (Line 3) Invalid input for function `np.arange()`.

   C. (Line 6) Arrays cannot be initialized using boolean values.

   D. (Line 7) Boolean array does not match the indexed array.

4. Which of the following array declarations will cause an error?

```
1  import numpy as np
2
3  a = np.zeros(3).astype(float)
4  b = np.zeros(3, 3)
5  c = np.ones(5, dtype=bool)
6  d = np.ones((5, 5))
```

   A. (Line 5) Array `c`

   B. (Line 6) Array `d`

   C. (Line 3) Array `a`

   D. (Line 4) Array `b`

5. Which of the following lines will raise an error?

```
1 data = ("apple", ["x", "y"], ("red", "green"))
2
3 data[1].append("z")          # Line 2
4 print(data[2][1])            # Line 3
5 data[0][2] = "p"             # Line 4
```

   A. Line 4.

   B. None of them.

   C. Line 2.

   D. Line 3.

6. What is going to be the final value of “x”?

```
1  import numpy as np
2
3  x = np.array(
4    [[1, 2],
5    [3, 4]]
6  )
7  x[1, :] = 0
```

   A. $\begin{bmatrix} 1 & 2 \\ 0 & 4 \end{bmatrix}$

   B. $\begin{bmatrix} 1 & 0 \\ 3 & 0 \end{bmatrix}$

   C. $\begin{bmatrix} 0 & 2 \\ 1 & 4 \end{bmatrix}$

   D. $\begin{bmatrix} 1 & 2 \\ 0 & 0 \end{bmatrix}$

Correct answers for Questions 1–6:

Question	Key
1	B
2	D
3	D
4	D
5	A
6	D

7. This function is designed to remove duplicate elements from a guest list for each event organized by a specific agency. Everything is contained in a dictionary where the keys are the events and the values are the guest lists. However, it does not remove them; it returns the same dictionary. Identify the problem(s) and fix the code.

```
def remove_duplicate_guests(agency_info: dict)->dict[list[str]]:
  for event, guests in agency_info.items():
    guests_removed_dupl = tuple(guests)

    agency_info[event] = guests

    return agency_info

agency_info = {
    "Summer Gala": [
        "Alice Johnson", "Mark Davis", "Sophia Lee",
        "Alice Johnson", "Mark Davis"
    ],
    "Tech Conference": [
        "Daniel Kim", "Olivia Brown", "Emma Wilson",
        "Daniel Kim", "Emma Wilson"
    ]
}

print(remove_duplicate_guests(agency_info))
#Expected Ouput: {"Summer Gala": ["Alice Johnson", "Mark Davis", "Sophia
    Lee"],
#"Tech Conference": ["Daniel Kim", "Olivia Brown", "Emma Wilson"]}

#Ouput: {"Summer Gala": ["Alice Johnson", "Mark Davis", "Sophia Lee", "
    Alice Johnson", "Mark Davis"],
#"Tech Conference": ["Daniel Kim", "Olivia Brown", "Emma Wilson", "Daniel
     Kim", "Emma Wilson"]}
```

Answer:

The first problem is that `tuple(guests)` does not remove duplicates, it just converts the list into a tuple that keeps every element. To remove duplicates we must use a set, and then turn it back into a list: `list(set(guests))`. *(+4 points)*

The second problem is that the result is stored in `guests_removed_dupl`, but the next line writes the original `guests` back into the dictionary, so nothing changes. We have to assign the cleaned list: `agency_info[event] = list(set(guests))`. *(+3 points)*

The third problem is that `return agency_info` is indented inside the `for` loop, so the function returns after cleaning only the first event. The `return` must be placed after the loop. *(+3 points)*

Comment: A set does not keep the order of the elements. If we need the guests to stay in their original order, as in the expected output, we can use `list(dict.fromkeys(guests))` instead.

(Optional, fixed code)

```
def remove_duplicate_guests(agency_info: dict) -> dict:
    for event, guests in agency_info.items():
        guests_removed_dupl = list(set(guests))
```

```
        agency_info[event] = guests_removed_dupl
    return agency_info
```

8. The following function should mutliply the elements of an array by 2. However the code shows an unexpected result. Explain the issue and fix the code.

```
import numpy as np

def double_array(arr: np.array) -> np.array:
        # Multiply each element by 2
        arr_double = arr * 2
        return arr_double

print(double_array([1, 2, 3]))
# Expected output: [2, 4, 6]
# Actual output:   [1, 2, 3, 1, 2, 3]
```

How would `double_array()` work on NumPy arrays of 2 or more dimensions?

Answer:

The function receives a Python **list**, not a NumPy array. Type hints do not convert anything, and the operator `*` on a list means repetition, not an element-wise product. This is why `[1, 2, 3] * 2` gives `[1, 2, 3, 1, 2, 3]`. *(+4 points)*

We need to turn the input into a NumPy array, either inside the function with `arr = np.array(arr)` before the multiplication, or by calling it with `double_array(np.array([1, 2, 3]))`. *(+3 points)*

For arrays of 2 or more dimensions, it works exactly the same: `arr * 2` is an element-wise operation, so it multiplies every single element by 2 and keeps the shape of the array. For instance, `[[1, 2], [3, 4]]` becomes `[[2, 4], [6, 8]]`. *(+3 points)*

(Optional, fixed code)

```
import numpy as np

def double_array(arr: np.array) -> np.array:
    # Make sure we work with a NumPy array
    arr = np.array(arr)
    arr_double = arr * 2
    return arr_double

print(double_array([1, 2, 3]))  # [2 4 6]
```

9. The following code is intended to replace all negative numbers in the array "`data`" with their absolute values using a boolean mask. However, the code raises an error `ValueError: NumPy boolean array indexing assignment cannot assign 7 input values to the 3 output values where the mask is true`. Explain the issue and fix the code.

```
import numpy as np

data = np.array([5, -3, 7, -1, 0, -6, 8])
mask = data < 0
data[mask] = -data
print(data)
# Expected output: [5, 3, 7, 1, 0, 6, 8]
# Actual output: ValueError
```

Answer:

The mask `data < 0` has only 3 `True` values (the positions of -3, -1 and -6), so `data[mask]` selects 3 elements. On the right side, however, we are assigning `-data`, which has all 7 elements, and NumPy cannot fit 7 values into 3 positions. *(+5 points)*

We must also select the values with the mask on the right side, so that both sides have the same number of elements: `data[mask] = -data[mask]`. *(+5 points)*

(Optional, fixed code)

```
import numpy as np

data = np.array([5, -3, 7, -1, 0, -6, 8])
mask = data < 0
data[mask] = -data[mask]
print(data)  # [5 3 7 1 0 6 8]
```

10. During your class's culinary trip around Spain, you have a list of foods to be tasted in each city. However, some classmates have dietary restrictions or preferences and have listed foods they won't eat. Your task is to accommodate everyone's preferences as much as possible.

    Create a function `foods_for_all()`. It takes two inputs:

    - `foods_by_city` a dictionary with city names and their food lists.
    - `food_restrictions` a list of tuples with students' names and the foods they avoid.

    The function `foods_for_all()` returns a dictionary with each city's food options that all students can eat.

    If a city ends up with no options for everyone, it is not included in the output.

    Also, ensure the original inputs remain unchanged after running your function.

```python
# Sample input
# Remember: your function should work with any other possibility!
foods_by_city = {
    "Madrid": ["Squid", "Omelette", "Churros", "Soup"],
    "Valencia": ["Paella", "Horchata", "Fartons"],
    "Cadiz": ["Fried fish", "Potato salad", "Prawns"]}

food_restrictions = [
("Alice", ["Squid", "Paella", "Prawns"]),
("Bob", ["Churros", "Fartons"]),
("Charlie", ["Prawns"]),
("Diana", ["Fried fish", "Paella"]),
("Eve", ["Squid", "Fartons"]),
("Fred", ["Churros", "Omelette", "Fartons"]),
("Gwen", ["Fried fish", "Prawns"]),
("Hilda", ["Squid", "Potato salad"]),
]
```

Answer:

We start by collecting every food that at least one student cannot eat. We create an empty set, `avoided = set()`, and loop over the tuples with `for name, foods in food_restrictions`. We ignore the name, and add the foods to the set with `avoided.update(foods)`. A set is a good choice because it avoids duplicates (many students avoid the same food) and checking `in` is fast. *(+6 points)*

Then we create an empty dictionary for the output, and loop over the cities with `for city, foods in foods_by_city.items()`. For each city we build a new list with only the foods that are not in `avoided`. *(+6 points)*

If this new list is empty, the city is skipped. Otherwise, we store the list in the output dictionary using the city name as key. *(+4 points)*

We never modify the input dictionary or its lists: the allowed foods go into new lists, and the restrictions are only read. Therefore, the original inputs remain unchanged. *(+4 points)*

(Optional, code)

```python
def foods_for_all(foods_by_city: dict, food_restrictions: list) -> dict:
    # Foods that at least one student avoids
    avoided = set()
    for name, foods in food_restrictions:
        avoided.update(foods)

    # Keep the foods that everybody can eat
    output = {}
    for city, foods in foods_by_city.items():
        allowed = []
        for food in foods:
            if food not in avoided:
                allowed.append(food)
        # Cities without options are not included
        if len(allowed) > 0:
            output[city] = allowed
    return output

# For the example: {"Madrid": ["Soup"], "Valencia": ["Horchata"]}
```

11. You are given a list of player names and a fixed number of steps X. Each player begins at position 0 and performs a random walk of X steps. Write a function `random_walks()` that simulates this process. At each step, the player should either move $+1$ or -1, chosen uniformly at random. After completing the walk for each player, return a dictionary where each key is a player's name, and the corresponding value is that player's final position.

    Your function should accept the following arguments:

    - `players` (positional): A list or tuple of player names.
    - `num_steps` (keyword): The number of steps for each player's random walk (same number for all of them), defaulting to 10.
    - `random_seed` (keyword): If provided, set the random seed to ensure reproducible results.

    Additionally, if a player's name appears multiple times in the input list, append a "-" to subsequent occurrences to ensure that all dictionary keys are unique. For example, if "Alex" appears twice, the keys should be "Alex" and "Alex−". If "Bob" appears three times, the keys should be "Bob", "Bob−", and "Bob−−".

    See the code below for an example of input and output:

```
players = ["Alex", "Alex", "Bob", "Bob", "Bob", "Chia"]
x = 10

print(random_walks(players, num_steps=x))
# Example output (actual results will vary due to randomness):
# {"Alex": 2, "Alex-": 0, "Bob": -1, "Bob-": -3, "Bob--": 0, "Chia": 4}
```

Answer:

The function receives the list of players as a positional argument, and the other two as keyword arguments with default values: `def random_walks(players, num_steps=10, random_seed=None)`. If a seed was provided, we set it at the beginning of the function with `np.random.seed(random_seed)`. We must check `if random_seed is not None`, because 0 is also a valid seed. *(+5 points)*

We initialize an empty dictionary `results = {}` and loop over the players. For each player we simulate the walk: we generate `num_steps` random steps, each of them either -1 or $+1$, using `np.random.choice([-1, 1], size=num_steps)`. *(+5 points)*

Because the player starts at position 0, the final position is simply the sum of all the steps, `steps.sum()`. We store it in the dictionary using the player's name as key, and after the loop we return the dictionary. *(+5 points)*

To keep the keys unique, before storing the result we check whether the name is already a key of the dictionary. While it is, we append a "-" to the name. This way, the second "Bob" becomes "Bob-" and the third one becomes "Bob–". *(+5 points)*

(Optional, code)

```
import numpy as np

def random_walks(players, num_steps=10, random_seed=None):
```

```
    # Set the seed only if the user gave one
    if random_seed is not None:
        np.random.seed(random_seed)
    results = {}
    for player in players:
        # Random steps of -1 or +1, and final position
        steps = np.random.choice([-1, 1], size=num_steps)
        final_position = int(steps.sum())
        # Make the key unique
        name = player
        while name in results:
            name = name + "-"
        results[name] = final_position
    return results
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.lower()` returns a copy of the string, all in lowercase.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.
- `s.replace(<pattern>, <new>)` returns a new string where all instances of `<pattern>` have been replaced with `<new>`.

Given a list named `ls`:

- `ls.append(element)` add element to that list.

Given a list or tuple named `ls`:

- `ls.index(element)` returns the index of element in that list.
- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` or (`a | b`) returns the union of set `a` with another set `b`.
- `a.intersection(b)` or (`a & b`) returns the intersection of set `a` with another set `b`.
- `a.difference(b)` or (`a - b`) returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.
- `input(msg)` is used to take user input, in form of a string.

NumPy Cheatsheet

Importing NumPy:

- `import numpy as np`
- `rng = np.random.default_rng()`

Creating arrays:

- `np.zeros((m, n))` creates an $m \times n$ array filled with zeros.
- `np.ones((m, n))` creates an $m \times n$ array filled with ones.
- `np.arange(start, stop, step)` creates values from `start` to `stop` (exclusive) with a given `step`.
- `np.linspace(start, stop, num)` creates `num` evenly spaced values between `start` and `stop`.

Given a NumPy array `arr`:

- `arr.ndim` returns the number of array dimensions, as a number.
- `arr.shape` returns the shape of an array, as a tuple.

Array operations:

- `np.dot(a, b)` computes the dot product.
- `np.matmul(a, b)` or `a @ b` performs matrix multiplication.
- `np.transpose(a)` or `a.T` transposes the array.

Array manipulation:

- `a.reshape((m, n))` changes the shape of an array.
- `a.flatten()` flattens a multidimensional array into 1D.
- `np.concatenate((a, b), axis=0)` joins arrays along an axis.

- `np.stack(<sequence>)` joins a sequence of arrays along a new axis.
- `np.vstack((a, b))`, `np.hstack((a, b))` vertical/horizontal stacking.
- `np.split(a, n)` splits an array into `n` parts.

Statistics and descriptive analysis:

- `np.mean(a)` mean of all elements.
- `np.median(a)` median of all elements.
- `np.std(a)` standard deviation.
- `np.var(a)` variance.
- `np.min(a)`, `np.max(a)` minimum and maximum.
- `np.argmin(a)`, `np.argmax(a)` indices of minimum and maximum.
- `np.corrcoef(a, b)` correlation coefficient matrix.
- `np.cov(a, b)` covariance matrix.

Linear algebra:

- `np.linalg.inv(a)` inverse of matrix `a`.
- `np.linalg.det(a)` determinant of `a`.
- `np.linalg.eig(a)` eigenvalues and eigenvectors.
- `np.linalg.norm(a)` vector or matrix norm.
- `np.linalg.solve(a, b)` solves $Ax = b$.

Logical operations and conditions:

- `np.where(cond, x, y)` selects elements from `x` or `y` based on condition.
- `np.all(a)`, `np.any(a)` check if all/any elements are True.
- `np.unique(a)` returns sorted unique elements.

Random number generation (using `rng`):

- `rng.random(size)` uniform distribution in $[0, 1)$.
- `rng.integers(low, high, size)` random integers in $[low, high)$.
- `rng.normal(mean, std, size)` normal (Gaussian) distribution.
- `rng.uniform(low, high, size)` uniform distribution in $[low, high)$.
- `rng.choice(a, size)` random sample from array `a`.

Random number generation (legacy API):

- `np.random.randint(low, high, size)` returns random integers from low (inclusive) to high (exclusive).
- `np.random.uniform(low, high, size)` samples from a uniform distribution.
- `np.random.seed(seed)` fixes the seed. After a seed reset, the same set of numbers will appear every time.

Math and trigonometry:

- `np.power(arr1, arr2)` returns the first array elements raised to powers from second array, element-wise.
- `np.sin(arr)` returns the trigonometric sine, assuming angle in radians.
- `np.cos(arr)` returns the trigonometric cosine, assuming angle in radians.
- `np.radians(arr)` converts angles from degrees to radians.
- `np.degrees(arr)` converts angles from radians to degrees.